```c
//###########################################################################
// FILE:   HWstarter_main.c
//
// TITLE:  HW Starter
//###########################################################################

// Included Files
#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>
#include <string.h>
#include <math.h>
#include <limits.h>
#include "F28x_Project.h"
#include "driverlib.h"
#include "device.h"
#include "F28379dSerial.h"
#include "LEDPatterns.h"
#include "song.h"
#include "dsp.h"
#include "fpu32/fpu_rfft.h"

#define PI          3.14159265358979323846264338327795
#define TWOPI       6.28318530717958647692528676655 9
#define HALFPI      1.57079632679489661923132169163 98
// The Launchpad's CPU Frequency set to 200 you should not change this value
#define LAUNCHPAD_CPU_FREQUENCY 200


// Interrupt Service Routines predefinition
__interrupt void cpu_timer0_isr(void);
__interrupt void cpu_timer1_isr(void);
__interrupt void cpu_timer2_isr(void);
__interrupt void SWI_isr(void);

float saturate(float, float);

// Count variables
uint32_t numTimer0calls = 0;
uint32_t numSWIcalls = 0;
extern uint32_t numRXA;
uint16_t UARTPrint = 0;
uint16_t LEDdisplaynum = 0;


// Exercise 8 global variables
float sinvalue = 0;
float time = 0;
float ampl = 3.0;
float frequency = 0.05;
float offset = 0.25;
int32_t timeint = 0;
float satvalue = 0;
#define SAT_LIMIT 2.65

void main(void)
{
    // PLL, WatchDog, enable Peripheral Clocks
    // This example function is found in the F2837xD_SysCtrl.c file.
    InitSysCtrl();

    InitGpio();

        // Blue LED on LaunchPad
    GPIO_SetupPinMux(31, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(31, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPASET.bit.GPIO31 = 1;

        // Red LED on LaunchPad
    GPIO_SetupPinMux(34, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(34, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPBSET.bit.GPIO34 = 1;

        // LED1 and PWM Pin
    GPIO_SetupPinMux(22, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(22, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;

        // LED2
    GPIO_SetupPinMux(94, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(94, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;

        // LED3
    GPIO_SetupPinMux(95, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(95, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;

        // LED4
    GPIO_SetupPinMux(97, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(97, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;

        // LED5
    GPIO_SetupPinMux(111, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(111, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;

        // LED6
    GPIO_SetupPinMux(130, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(130, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPECLEAR.bit.GPIO130 = 1;

        // LED7
    GPIO_SetupPinMux(131, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(131, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPECLEAR.bit.GPIO131 = 1;

        // LED8
    GPIO_SetupPinMux(25, GPIO_MUX_CPU1, 0);
```

```
    GPIO_SetupPinOptions(25, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPACLEAR.bit.GPIO25 = 1;

        // LED9
    GPIO_SetupPinMux(26, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(26, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPACLEAR.bit.GPIO26 = 1;

        // LED10
    GPIO_SetupPinMux(27, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(27, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPACLEAR.bit.GPIO27 = 1;

        // LED11
    GPIO_SetupPinMux(60, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(60, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPBCLEAR.bit.GPIO60 = 1;

        // LED12
    GPIO_SetupPinMux(61, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(61, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;

        // LED13
    GPIO_SetupPinMux(157, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(157, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPECLEAR.bit.GPIO157 = 1;

        // LED14
    GPIO_SetupPinMux(158, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(158, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPECLEAR.bit.GPIO158 = 1;

        // LED15
    GPIO_SetupPinMux(159, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(159, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPECLEAR.bit.GPIO159 = 1;

        // LED16
    GPIO_SetupPinMux(160, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(160, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPFCLEAR.bit.GPIO160 = 1;

    //WIZNET Reset
    GPIO_SetupPinMux(0, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(0, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPASET.bit.GPIO0 = 1;

    //ESP8266 Reset
    GPIO_SetupPinMux(1, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(1, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPASET.bit.GPIO1 = 1;

        //SPIRAM  CS  Chip Select
    GPIO_SetupPinMux(19, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(19, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPASET.bit.GPIO19 = 1;

    //DRV8874 #1 DIR  Direction
    GPIO_SetupPinMux(29, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(29, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPASET.bit.GPIO29 = 1;

    //DRV8874 #2 DIR  Direction
    GPIO_SetupPinMux(32, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(32, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPBSET.bit.GPIO32 = 1;

    //DAN28027  CS  Chip Select
    GPIO_SetupPinMux(9, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(9, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPASET.bit.GPIO9 = 1;

    //MPU9250  CS  Chip Select
    GPIO_SetupPinMux(66, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(66, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPCSET.bit.GPIO66 = 1;

        //WIZNET  CS  Chip Select
    GPIO_SetupPinMux(125, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(125, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPDSET.bit.GPIO125 = 1;

    //PushButton 1
    GPIO_SetupPinMux(4, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(4, GPIO_INPUT, GPIO_PULLUP);

    //PushButton 2
    GPIO_SetupPinMux(5, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(5, GPIO_INPUT, GPIO_PULLUP);

    //PushButton 3
    GPIO_SetupPinMux(6, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(6, GPIO_INPUT, GPIO_PULLUP);

    //PushButton 4
    GPIO_SetupPinMux(7, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(7, GPIO_INPUT, GPIO_PULLUP);

        //Joy Stick Pushbutton
    GPIO_SetupPinMux(8, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(8, GPIO_INPUT, GPIO_PULLUP);

    // Clear all interrupts and initialize PIE vector table:
    // Disable CPU interrupts
    DINT;

    // Initialize the PIE control registers to their default state.
    // The default state is all PIE interrupts disabled and flags
    // are cleared.
    // This function is found in the F2837xD_PieCtrl.c file.
    InitPieCtrl();
```

```
223
224     // Disable CPU interrupts and clear all CPU interrupt flags:
225     IER = 0x0000;
226     IFR = 0x0000;
227
228     // Initialize the PIE vector table with pointers to the shell Interrupt
229     // Service Routines (ISR).
230     // This will populate the entire table, even if the interrupt
231     // is not used in this example.  This is useful for debug purposes.
232     // The shell ISR routines are found in F2837xD_DefaultIsr.c.
233     // This function is found in F2837xD_PieVect.c.
234     InitPieVectTable();
235
236     // Interrupts that are used in this example are re-mapped to
237     // ISR functions found within this project
238     EALLOW;  // This is needed to write to EALLOW protected registers
239     PieVectTable.TIMER0_INT = &cpu_timer0_isr;
240     PieVectTable.TIMER1_INT = &cpu_timer1_isr;
241     PieVectTable.TIMER2_INT = &cpu_timer2_isr;
242     PieVectTable.SCIA_RX_INT = &RXAINT_recv_ready;
243     PieVectTable.SCIB_RX_INT = &RXBINT_recv_ready;
244     PieVectTable.SCIC_RX_INT = &RXCINT_recv_ready;
245     PieVectTable.SCID_RX_INT = &RXDINT_recv_ready;
246     PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
247     PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
248     PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
249     PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
250
251     PieVectTable.EMIF_ERROR_INT = &SWI_isr;
252     EDIS;    // This is needed to disable write to EALLOW protected registers
253
254
255     // Initialize the CpuTimers Device Peripheral. This function can be
256     // found in F2837xD_CpuTimers.c
257     InitCpuTimers();
258
259     // Configure CPU-Timer 0, 1, and 2 to interrupt every given period:
260     // 200MHz CPU Freq,                     Period (in uSeconds)
261     ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000);
262     ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 20000);
263     ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 5000);
264
265     // Enable CpuTimer Interrupt bit TIE
266     CpuTimer0Regs.TCR.all = 0x4000;
267     CpuTimer1Regs.TCR.all = 0x4000;
268     CpuTimer2Regs.TCR.all = 0x4000;
269
270     init_serialSCIA(&SerialA,115200);
271     //    init_serialSCIC(&SerialC,115200);
272     //    init_serialSCID(&SerialD,115200);
273
274
275     // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
276     // which is connected to CPU-Timer 1, and CPU int 14, which is connected
277     // to CPU-Timer 2:  int 12 is for the SWI.
278     IER |= M_INT1;
279     IER |= M_INT8;  // SCIC SCID
280     IER |= M_INT9;  // SCIA
281     IER |= M_INT12;
282     IER |= M_INT13;
283     IER |= M_INT14;
284
285     // Enable TINT0 in the PIE: Group 1 interrupt 7
286     PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
287         // Enable SWI in the PIE: Group 12 interrupt 9
288     PieCtrlRegs.PIEIER12.bit.INTx9 = 1;
289
290     // Enable global Interrupts and higher priority real-time debug events
291     EINT;  // Enable Global interrupt INTM
292     ERTM;  // Enable Global realtime interrupt DBGM
293
294
295     // IDLE loop. Just sit and loop forever (optional):
296     while(1)
297     {
298         if (UARTPrint == 1 ) {
299             //            Excercise 8
300             ++timeint;
301             time = timeint * 0.25;
302             sinvalue = ampl*sin(2 * PI * frequency * time) + offset;
303
304             satvalue = saturate(sinvalue, SAT_LIMIT);
305             serial_printf(&SerialA,"Timeint = %ld (wrong: %d), Time = %.2fsec, Input = %.3f, SatOut = %.2f\r\n",timeint,timeint, time, sinvalue, satvalue);
306
307 //            serial_printf(&SerialA,"Num Timer2:%ld Num SerialRX: %ld\r\n",CpuTimer2.InterruptCount,numRXA);
308             UARTPrint = 0;
309         }
310     }
311 }
312
313 float saturate(float input, float saturation_limit){
314     if(input > saturation_limit){
315         return saturation_limit;
316     }
317     else if (input < saturation_limit){
318         return -saturation_limit;
319     }
320     return input;
321 }
322
323
324
325
326 // SWI_isr,  Using this interrupt as a Software started interrupt
327 __interrupt void SWI_isr(void) {
328
329     // These three lines of code allow SWI_isr, to be interrupted by other interrupt functions
330         // making it lower priority than all other Hardware interrupts.
331         PieCtrlRegs.PIEACK.all = PIEACK_GROUP12;
332     asm("       NOP");                  // Wait one cycle
333     EINT;                               // Clear INTM to enable interrupts
334
```

```
335
336
337     // Insert SWI ISR Code here.......
338
339
340     numSWIcalls++;
341
342     DINT;
343
344 }
345
346 // cpu_timer0_isr - CPU Timer0 ISR
347 __interrupt void cpu_timer0_isr(void)
348 {
349     CpuTimer0.InterruptCount++;
350
351     numTimer0calls++;
352
353 //    if ((numTimer0calls%50) == 0) {
354 //        PieCtrlRegs.PIEIFR12.bit.INTx9 = 1;  // Manually cause the interrupt for the SWI
355 //    }
356
357     if ((numTimer0calls%250) == 0) {
358 //        displayLEDletter(LEDdisplaynum);
359         LEDdisplaynum++;
360         if (LEDdisplaynum == 0xFFFF) {  // prevent roll over exception
361             LEDdisplaynum = 0;
362         }
363     }
364
365         // Blink LaunchPad Red LED
366 //    GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1;
367
368     // Acknowledge this interrupt to receive more interrupts from group 1
369     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
370 }
371
372 // cpu_timer1_isr - CPU Timer1 ISR
373 __interrupt void cpu_timer1_isr(void)
374 {
375
376
377     CpuTimer1.InterruptCount++; // MJ Increments the timer's internal interrupt counter
378 }
379
380 uint16_t prev_button_1 = 0;
381 uint16_t prev_button_4 = 0;
382
383 // cpu_timer2_isr CPU Timer2 ISR
384 __interrupt void cpu_timer2_isr(void)
385 {
386
387
388         // Blink LaunchPad Blue LED
389     GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;
390
391     CpuTimer2.InterruptCount++;
392
393         if ((CpuTimer2.InterruptCount % 50) == 0) {
394                 UARTPrint = 1;
395         }
396
397         if((CpuTimer2.InterruptCount % 20) == 0){
398             GpioDataRegs.GPBTOGGLE.bit.GPIO60 = 1; // Toggle LED 11
399             GpioDataRegs.GPATOGGLE.bit.GPIO27 = 1; // Toggle LED 10
400         }
401
402         uint16_t button_1 = GpioDataRegs.GPADAT.bit.GPIO4;
403         uint16_t button_4 = GpioDataRegs.GPADAT.bit.GPIO7;
404
405         if(button_1 == 1  && prev_button_1 == 0){ //If Button 1 is pressed
406                 // Toggle LED 12 and 13
407         GpioDataRegs.GPBTOGGLE.bit.GPIO61 = 1; // Toggle LED 12
408         GpioDataRegs.GPETOGGLE.bit.GPIO157 = 1; // Toggle LED 13
409         }
410         prev_button_1 = button_1;
411
412     if(button_4 == 1  && prev_button_4 == 0){ //If Button 4 is pressed
413             // Toggle LED 14 and 15
414         GpioDataRegs.GPETOGGLE.bit.GPIO159 = 1; // Toggle LED 14
415         GpioDataRegs.GPFTOGGLE.bit.GPIO160 = 1; // Toggle LED 15
416     }
417     prev_button_4 = button_4;
418 }
```

```c
/* SERIAL.C: This code is designed to act as a low-level serial driver for
    higher-level programming.  Ideally, one could simply call init_serial()
    to initialize the serial port, then use serial_send("data", 4) to send
    an array of data (8-bit unsigned character strings).

    WRITTEN BY : Paul Miller <pamiller@uiuc.edu>
    $Id: serial.c,v 1.4 2003/08/08 16:08:56 paul Exp $
 */
#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>
#include <string.h>
#include <math.h>
#include <limits.h>

#include "F28x_Project.h"     // Device Headerfile and Examples Include File
#include <buffer.h>
#include <F28379dSerial.h>
#include <F2837xD_sci.h>

serialSCIA_t SerialA;
serialSCIB_t SerialB;
serialSCIC_t SerialC;
serialSCID_t SerialD;

char RXAdata = 0;
char RXBdata = 0;
char RXCdata = 0;
char RXDdata = 0;
uint32_t numRXA = 0;
uint32_t numRXB = 0;
uint32_t numRXC = 0;
uint32_t numRXD = 0;


// for SerialA
uint16_t init_serialSCIA(serialSCIA_t *s, uint32_t baud)
{
    volatile struct SCI_REGS *sci;
    uint32_t clk;

    if (s == &SerialA) {
        sci = &SciaRegs;
        s->sci = sci;
        init_bufferSCIA(&s->TX);

        GPIO_SetupPinMux(43, GPIO_MUX_CPU1, 15);
        GPIO_SetupPinOptions(43, GPIO_INPUT, GPIO_PULLUP);
        GPIO_SetupPinMux(42, GPIO_MUX_CPU1, 15);
        GPIO_SetupPinOptions(42, GPIO_OUTPUT, GPIO_PUSHPULL);

    } else {
        return 1;
    }

    /* init for standard baud,8N1 comm */
    sci->SCICTL1.bit.SWRESET = 0;       // init SCI state machines and opt flags
    sci->SCICCR.all = 0x0;
    sci->SCICTL1.all = 0x0;
    sci->SCICTL2.all = 0x0;
    sci->SCIPRI.all = 0x0;
    clk = LSPCLK_HZ;                    // set baud rate
    clk /= baud*8;
    clk--;
    sci->SCILBAUD.all = clk & 0xFF;
    sci->SCIHBAUD.all = (clk >> 8) & 0xFF;

    sci->SCICCR.bit.SCICHAR = 0x7;      // (8) 8 bits per character
    sci->SCICCR.bit.PARITYENA = 0;      // (N) disable party calculation
    sci->SCICCR.bit.STOPBITS = 0;       // (1) transmit 1 stop bit
    sci->SCICCR.bit.LOOPBKENA = 0;      // disable loopback test
    sci->SCICCR.bit.ADDRIDLE_MODE = 0;  // idle-line mode (non-multiprocessor SCI comm)

    sci->SCIFFCT.bit.FFTXDLY = 0;       // TX: zero-delay

    sci->SCIFFTX.bit.SCIFFENA = 1;      // enable SCI fifo enhancements
    sci->SCIFFTX.bit.TXFIFORESET = 0;
    sci->SCIFFTX.bit.TXFFIL = 0x0;// TX: fifo interrupt at all levels   ???? is this correct
    sci->SCIFFTX.bit.TXFFINTCLR = 1;  // TX: clear interrupt flag
    sci->SCIFFTX.bit.TXFFIENA = 0;      // TX: disable fifo interrupt
    sci->SCIFFTX.bit.TXFIFORESET = 1;

    sci->SCIFFRX.bit.RXFIFORESET = 0;   // RX: fifo reset
    sci->SCIFFRX.bit.RXFFINTCLR = 1;    // RX: clear interrupt flag
    sci->SCIFFRX.bit.RXFFIENA = 1;      // RX: enable fifo interrupt
    sci->SCIFFRX.bit.RXFFIL = 0x1;      // RX: fifo interrupt
```

```
    sci->SCIFFRX.bit.RXFIFORESET = 1;   // RX: re-enable fifo

    sci->SCICTL2.bit.RXBKINTENA = 0;    // disable receiver/error interrupt
    sci->SCICTL2.bit.TXINTENA = 0;      // disable transmitter interrupt

    sci->SCICTL1.bit.TXWAKE = 0;
    sci->SCICTL1.bit.SLEEP = 0;         // disable sleep mode
    sci->SCICTL1.bit.RXENA = 1;         // enable SCI receiver
    sci->SCICTL1.bit.RXERRINTENA = 0;   // disable receive error interrupt
    sci->SCICTL1.bit.TXENA = 1;         // enable SCI transmitter
    sci->SCICTL1.bit.SWRESET = 1;       // re-enable SCI

    /* enable PIE interrupts */
    if (s == &SerialA) {
        PieCtrlRegs.PIEIER9.bit.INTx1 = 1;
        PieCtrlRegs.PIEIER9.bit.INTx2 = 1;
        IER |= (M_INT9);
        PieCtrlRegs.PIEACK.all = (PIEACK_GROUP9);

    }

    return 0;
}

void uninit_serialSCIA(serialSCIA_t *s)
{
    volatile struct SCI_REGS *sci = s->sci;

    /* disable PIE interrupts */
    if (s == &SerialA) {
        PieCtrlRegs.PIEIER9.bit.INTx1 = 0;
        PieCtrlRegs.PIEIER9.bit.INTx2 = 0;
        IER &= ~M_INT9;
    }
    sci->SCICTL1.bit.RXERRINTENA = 0;   // disable receive error interrupt
    sci->SCICTL2.bit.RXBKINTENA = 0;    // disable receiver/error interrupt
    sci->SCICTL2.bit.TXINTENA = 0;      // disable transmitter interrupt

    sci->SCICTL1.bit.RXENA = 0;         // disable SCI receiver
    sci->SCICTL1.bit.TXENA = 0;         // disable SCI transmitter
}



/**************************************************************************
 * SERIAL_SEND()
 *
 * "User level" function to send data via serial.  Return value is the
 * length of data successfully copied to the TX buffer.
 *************************************************************************/
uint16_t serial_sendSCIA(serialSCIA_t *s, char *data, uint16_t len)
{
    uint16_t i = 0;
    if (len && s->TX.size < BUF_SIZESCIA) {
        for (i = 0; i < len; i++) {
            if (buf_writeSCIA_1(&s->TX, data[i] & 0x00FF) != 0) break;
        }
        s->sci->SCIFFTX.bit.TXFFINTCLR = 1;  // TX: clear interrupt flag
        s->sci->SCIFFTX.bit.TXFFIENA = 1;       // TX: enable fifo interrupt
    }
    return i;
}

// For SerialB
uint16_t init_serialSCIB(serialSCIB_t *s, uint32_t baud)
{
    volatile struct SCI_REGS *sci;
    uint32_t clk;

    if (s == &SerialB) {
        sci = &ScibRegs;
        s->sci = sci;
        init_bufferSCIB(&s->TX);

        GPIO_SetupPinMux(15, GPIO_MUX_CPU1, 2);
        GPIO_SetupPinOptions(15, GPIO_INPUT, GPIO_PULLUP);
        GPIO_SetupPinMux(14, GPIO_MUX_CPU1, 2);
        GPIO_SetupPinOptions(14, GPIO_OUTPUT, GPIO_PUSHPULL);


    } else {
        return 1;
    }

    /* init for standard baud,8N1 comm */
    sci->SCICTL1.bit.SWRESET = 0;       // init SCI state machines and opt flags
    sci->SCICCR.all = 0x0;
    sci->SCICTL1.all = 0x0;
    sci->SCICTL2.all = 0x0;
```

```
176     sci->SCIPRI.all = 0x0;
177     clk = LSPCLK_HZ;                    // set baud rate
178     clk /= baud*8;
179     clk--;
180     sci->SCILBAUD.all = clk & 0xFF;
181     sci->SCIHBAUD.all = (clk >> 8) & 0xFF;
182 
183     sci->SCICCR.bit.SCICHAR = 0x7;      // (8) 8 bits per character
184     sci->SCICCR.bit.PARITYENA = 0;      // (N) disable party calculation
185     sci->SCICCR.bit.STOPBITS = 0;       // (1) transmit 1 stop bit
186     sci->SCICCR.bit.LOOPBKENA = 0;      // disable loopback test
187     sci->SCICCR.bit.ADDRIDLE_MODE = 0;  // idle-line mode (non-multiprocessor SCI comm)
188 
189     sci->SCIFFCT.bit.FFTXDLY = 0;       // TX: zero-delay
190 
191     sci->SCIFFTX.bit.SCIFFENA = 1;      // enable SCI fifo enhancements
192     sci->SCIFFTX.bit.TXFIFORESET = 0;
193     sci->SCIFFTX.bit.TXFFIL = 0x0;// TX: fifo interrupt at all levels   ???? is this correct
194     sci->SCIFFTX.bit.TXFFINTCLR = 1;  // TX: clear interrupt flag
195     sci->SCIFFTX.bit.TXFFIENA = 0;      // TX: disable fifo interrupt
196     sci->SCIFFTX.bit.TXFIFORESET = 1;
197 
198     sci->SCIFFRX.bit.RXFIFORESET = 0;   // RX: fifo reset
199     sci->SCIFFRX.bit.RXFFINTCLR = 1;    // RX: clear interrupt flag
200     sci->SCIFFRX.bit.RXFFIENA = 1;      // RX: enable fifo interrupt
201     sci->SCIFFRX.bit.RXFFIL = 0x1;      // RX: fifo interrupt
202     sci->SCIFFRX.bit.RXFIFORESET = 1;   // RX: re-enable fifo
203 
204     sci->SCICTL2.bit.RXBKINTENA = 0;    // disable receiver/error interrupt
205     sci->SCICTL2.bit.TXINTENA = 0;      // disable transmitter interrupt
206 
207     sci->SCICTL1.bit.TXWAKE = 0;
208     sci->SCICTL1.bit.SLEEP = 0;         // disable sleep mode
209     sci->SCICTL1.bit.RXENA = 1;         // enable SCI receiver
210     sci->SCICTL1.bit.RXERRINTENA = 0;   // disable receive error interrupt
211     sci->SCICTL1.bit.TXENA = 1;         // enable SCI transmitter
212     sci->SCICTL1.bit.SWRESET = 1;       // re-enable SCI
213 
214     /* enable PIE interrupts */
215     if (s == &SerialB) {
216         PieCtrlRegs.PIEIER9.bit.INTx3 = 1;
217         PieCtrlRegs.PIEIER9.bit.INTx4 = 1;
218         IER |= (M_INT9);
219         PieCtrlRegs.PIEACK.all = (PIEACK_GROUP9);
220     }
221 
222     return 0;
223 }
224 
225 void uninit_serialSCIB(serialSCIB_t *s)
226 {
227     volatile struct SCI_REGS *sci = s->sci;
228 
229     /* disable PIE interrupts */
230     if (s == &SerialB) {
231         PieCtrlRegs.PIEIER9.bit.INTx3 = 0;
232         PieCtrlRegs.PIEIER9.bit.INTx4 = 0;
233         IER &= ~M_INT9;
234     }
235 
236     sci->SCICTL1.bit.RXERRINTENA = 0;   // disable receive error interrupt
237     sci->SCICTL2.bit.RXBKINTENA = 0;    // disable receiver/error interrupt
238     sci->SCICTL2.bit.TXINTENA = 0;      // disable transmitter interrupt
239 
240     sci->SCICTL1.bit.RXENA = 0;         // disable SCI receiver
241     sci->SCICTL1.bit.TXENA = 0;         // disable SCI transmitter
242 }
243 
244 
245 
246 /**********************************************************************
247  * SERIAL_SEND()
248  *
249  * "User level" function to send data via serial.  Return value is the
250  * length of data successfully copied to the TX buffer.
251  **********************************************************************/
252 uint16_t serial_sendSCIB(serialSCIB_t *s, char *data, uint16_t len)
253 {
254     uint16_t i = 0;
255     if (len && s->TX.size < BUF_SIZESCIB) {
256         for (i = 0; i < len; i++) {
257             if (buf_writeSCIB_1(&s->TX, data[i] & 0x00FF) != 0) break;
258         }
259         s->sci->SCIFFTX.bit.TXFFINTCLR = 1;  // TX: clear interrupt flag
260         s->sci->SCIFFTX.bit.TXFFIENA = 1;       // TX: enable fifo interrupt
261     }
262     return i;
263 }
264 
```

```
// for SerialC
uint16_t init_serialSCIC(serialSCIC_t *s, uint32_t baud)
{
    volatile struct SCI_REGS *sci;
    uint32_t clk;

    if (s == &SerialC) {
        sci = &ScicRegs;
        s->sci = sci;
        init_bufferSCIC(&s->TX);
        GPIO_SetupPinMux(139, GPIO_MUX_CPU1, 6);
        GPIO_SetupPinOptions(139, GPIO_INPUT, GPIO_PULLUP);
        GPIO_SetupPinMux(56, GPIO_MUX_CPU1, 6);
        GPIO_SetupPinOptions(56, GPIO_OUTPUT, GPIO_PUSHPULL);

    } else {
        return 1;
    }

    /* init for standard baud,8N1 comm */
    sci->SCICTL1.bit.SWRESET = 0;       // init SCI state machines and opt flags
    sci->SCICCR.all = 0x0;
    sci->SCICTL1.all = 0x0;
    sci->SCICTL2.all = 0x0;
    sci->SCIPRI.all = 0x0;
    clk = LSPCLK_HZ;                    // set baud rate
    clk /= baud*8;
    clk--;
    sci->SCILBAUD.all = clk & 0xFF;
    sci->SCIHBAUD.all = (clk >> 8) & 0xFF;

    sci->SCICCR.bit.SCICHAR = 0x7;      // (8) 8 bits per character
    sci->SCICCR.bit.PARITYENA = 0;      // (N) disable party calculation
    sci->SCICCR.bit.STOPBITS = 0;       // (1) transmit 1 stop bit
    sci->SCICCR.bit.LOOPBKENA = 0;      // disable loopback test
    sci->SCICCR.bit.ADDRIDLE_MODE = 0;  // idle-line mode (non-multiprocessor SCI comm)

    sci->SCIFFCT.bit.FFTXDLY = 0;       // TX: zero-delay

    sci->SCIFFTX.bit.SCIFFENA = 1;      // enable SCI fifo enhancements
    sci->SCIFFTX.bit.TXFIFORESET = 0;
    sci->SCIFFTX.bit.TXFFIL = 0x0;// TX: fifo interrupt at all levels   ???? is this correct
    sci->SCIFFTX.bit.TXFFINTCLR = 1;  // TX: clear interrupt flag
    sci->SCIFFTX.bit.TXFFIENA = 0;      // TX: disable fifo interrupt
    sci->SCIFFTX.bit.TXFIFORESET = 1;

    sci->SCIFFRX.bit.RXFIFORESET = 0;   // RX: fifo reset
    sci->SCIFFRX.bit.RXFFINTCLR = 1;    // RX: clear interrupt flag
    sci->SCIFFRX.bit.RXFFIENA = 1;      // RX: enable fifo interrupt
    sci->SCIFFRX.bit.RXFFIL = 0x1;      // RX: fifo interrupt
    sci->SCIFFRX.bit.RXFIFORESET = 1;   // RX: re-enable fifo

    sci->SCICTL2.bit.RXBKINTENA = 0;    // disable receiver/error interrupt
    sci->SCICTL2.bit.TXINTENA = 0;      // disable transmitter interrupt

    sci->SCICTL1.bit.TXWAKE = 0;
    sci->SCICTL1.bit.SLEEP = 0;         // disable sleep mode
    sci->SCICTL1.bit.RXENA = 1;         // enable SCI receiver
    sci->SCICTL1.bit.RXERRINTENA = 0;   // disable receive error interrupt
    sci->SCICTL1.bit.TXENA = 1;         // enable SCI transmitter
    sci->SCICTL1.bit.SWRESET = 1;       // re-enable SCI

    /* enable PIE interrupts */
    if (s == &SerialC) {
        PieCtrlRegs.PIEIER8.bit.INTx5 = 1;
        PieCtrlRegs.PIEIER8.bit.INTx6 = 1;
        PieCtrlRegs.PIEACK.all = (PIEACK_GROUP8);
        IER |= (M_INT8);

    }

    return 0;
}

void uninit_serialSCIC(serialSCIC_t *s)
{
    volatile struct SCI_REGS *sci = s->sci;

    /* disable PIE interrupts */
    if (s == &SerialC) {
        PieCtrlRegs.PIEIER8.bit.INTx5 = 0;
        PieCtrlRegs.PIEIER8.bit.INTx6 = 0;
        IER &= ~M_INT8;
    }

    sci->SCICTL1.bit.RXERRINTENA = 0;   // disable receive error interrupt
    sci->SCICTL2.bit.RXBKINTENA = 0;    // disable receiver/error interrupt
    sci->SCICTL2.bit.TXINTENA = 0;      // disable transmitter interrupt

```

```
354     sci->SCICTL1.bit.RXENA = 0;         // disable SCI receiver
355     sci->SCICTL1.bit.TXENA = 0;         // disable SCI transmitter
356 }
357
358
359
360 /**********************************************************************
361  * SERIAL_SEND()
362  *
363  * "User level" function to send data via serial.  Return value is the
364  * length of data successfully copied to the TX buffer.
365  **********************************************************************/
366 uint16_t serial_sendSCIC(serialSCIC_t *s, char *data, uint16_t len)
367 {
368     uint16_t i = 0;
369     if (len && s->TX.size < BUF_SIZESCIC) {
370         for (i = 0; i < len; i++) {
371             if (buf_writeSCIC_1(&s->TX, data[i] & 0x00FF) != 0) break;
372         }
373         s->sci->SCIFFTX.bit.TXFFINTCLR = 1;  // TX: clear interrupt flag
374         s->sci->SCIFFTX.bit.TXFFIENA = 1;       // TX: enable fifo interrupt
375     }
376     return i;
377 }
378
379 // For Serial D
380 uint16_t init_serialSCID(serialSCID_t *s, uint32_t baud)
381 {
382     volatile struct SCI_REGS *sci;
383     uint32_t clk;
384
385     if (s == &SerialD) {
386         sci = &ScidRegs;
387         s->sci = sci;
388         init_bufferSCID(&s->TX);
389         GPIO_SetupPinMux(105, GPIO_MUX_CPU1, 6);
390         GPIO_SetupPinOptions(105, GPIO_INPUT, GPIO_PULLUP);
391         GPIO_SetupPinMux(104, GPIO_MUX_CPU1, 6);
392         GPIO_SetupPinOptions(104, GPIO_OUTPUT, GPIO_PUSHPULL);
393     }
394     else {
395         return 1;
396     }
397
398     /* init for standard baud,8N1 comm */
399     sci->SCICTL1.bit.SWRESET = 0;       // init SCI state machines and opt flags
400     sci->SCICCR.all = 0x0;
401     sci->SCICTL1.all = 0x0;
402     sci->SCICTL2.all = 0x0;
403     sci->SCIPRI.all = 0x0;
404     clk = LSPCLK_HZ;                    // set baud rate
405     clk /= baud*8;
406     clk--;
407     sci->SCILBAUD.all = clk & 0xFF;
408     sci->SCIHBAUD.all = (clk >> 8) & 0xFF;
409
410     sci->SCICCR.bit.SCICHAR = 0x7;      // (8) 8 bits per character
411     sci->SCICCR.bit.PARITYENA = 0;      // (N) disable party calculation
412     sci->SCICCR.bit.STOPBITS = 0;       // (1) transmit 1 stop bit
413     sci->SCICCR.bit.LOOPBKENA = 0;      // disable loopback test
414     sci->SCICCR.bit.ADDRIDLE_MODE = 0;  // idle-line mode (non-multiprocessor SCI comm)
415
416     sci->SCIFFCT.bit.FFTXDLY = 0;       // TX: zero-delay
417
418     sci->SCIFFTX.bit.SCIFFENA = 1;      // enable SCI fifo enhancements
419     sci->SCIFFTX.bit.TXFIFORESET = 0;
420     sci->SCIFFTX.bit.TXFFIL = 0x0;// TX: fifo interrupt at all levels   ???? is this correct
421     sci->SCIFFTX.bit.TXFFINTCLR = 1;  // TX: clear interrupt flag
422     sci->SCIFFTX.bit.TXFFIENA = 0;      // TX: disable fifo interrupt
423     sci->SCIFFTX.bit.TXFIFORESET = 1;
424
425     sci->SCIFFRX.bit.RXFIFORESET = 0;   // RX: fifo reset
426     sci->SCIFFRX.bit.RXFFINTCLR = 1;    // RX: clear interrupt flag
427     sci->SCIFFRX.bit.RXFFIENA = 1;      // RX: enable fifo interrupt
428     sci->SCIFFRX.bit.RXFFIL = 0x1;      // RX: fifo interrupt
429     sci->SCIFFRX.bit.RXFIFORESET = 1;   // RX: re-enable fifo
430
431     sci->SCICTL2.bit.RXBKINTENA = 0;    // disable receiver/error interrupt
432     sci->SCICTL2.bit.TXINTENA = 0;      // disable transmitter interrupt
433
434     sci->SCICTL1.bit.TXWAKE = 0;
435     sci->SCICTL1.bit.SLEEP = 0;         // disable sleep mode
436     sci->SCICTL1.bit.RXENA = 1;         // enable SCI receiver
437     sci->SCICTL1.bit.RXERRINTENA = 0;   // disable receive error interrupt
438     sci->SCICTL1.bit.TXENA = 1;         // enable SCI transmitter
439     sci->SCICTL1.bit.SWRESET = 1;       // re-enable SCI
440
441     /* enable PIE interrupts */
442     if (s == &SerialD) {
```

```
443         PieCtrlRegs.PIEIER8.bit.INTx7 = 1;
444         PieCtrlRegs.PIEIER8.bit.INTx8 = 1;
445         PieCtrlRegs.PIEACK.all = (PIEACK_GROUP8);
446         IER |= (M_INT8);
447     }
448 
449     return 0;
450 }
451 
452 void uninit_serialSCID(serialSCID_t *s)
453 {
454     volatile struct SCI_REGS *sci = s->sci;
455 
456     /* disable PIE interrupts */
457     if (s == &SerialD) {
458         PieCtrlRegs.PIEIER8.bit.INTx7 = 0;
459         PieCtrlRegs.PIEIER8.bit.INTx8 = 0;
460         IER &= ~M_INT8;
461     }
462 
463     sci->SCICTL1.bit.RXERRINTENA = 0;   // disable receive error interrupt
464     sci->SCICTL2.bit.RXBKINTENA = 0;    // disable receiver/error interrupt
465     sci->SCICTL2.bit.TXINTENA = 0;      // disable transmitter interrupt
466 
467     sci->SCICTL1.bit.RXENA = 0;         // disable SCI receiver
468     sci->SCICTL1.bit.TXENA = 0;         // disable SCI transmitter
469 }
470 
471 
472 
473 /**********************************************************************
474  * SERIAL_SEND()
475  *
476  * "User level" function to send data via serial.  Return value is the
477  * length of data successfully copied to the TX buffer.
478  **********************************************************************/
479 uint16_t serial_sendSCID(serialSCID_t *s, char *data, uint16_t len)
480 {
481     uint16_t i = 0;
482     if (len && s->TX.size < BUF_SIZESCID) {
483         for (i = 0; i < len; i++) {
484             if (buf_writeSCID_1(&s->TX, data[i] & 0x00FF) != 0) break;
485         }
486         s->sci->SCIFFTX.bit.TXFFINTCLR = 1;  // TX: clear interrupt flag
487         s->sci->SCIFFTX.bit.TXFFIENA = 1;       // TX: enable fifo interrupt
488     }
489     return i;
490 }
491 
492 
493 
494 /**********************************************************************
495  * TXxINT_DATA_SENT()
496  *
497  * Executed when transmission is ready for additional data.  These functions
498  * read the next char of data and put it in the TXBUF register for transfer.
499  **********************************************************************/
500 #ifdef _FLASH
501 #pragma CODE_SECTION(TXAINT_data_sent, ".TI.ramfunc");
502 #endif
503 __interrupt void TXAINT_data_sent(void)
504 {
505     char data;
506     if (buf_readSCIA_1(&SerialA.TX,0,&data) == 0) {
507         while ( (buf_readSCIA_1(&SerialA.TX,0,&data) == 0)
508                 && (SerialA.sci->SCIFFTX.bit.TXFFST != 0x10) ) {
509             buf_removeSCIA(&SerialA.TX, 1);
510             SerialA.sci->SCITXBUF.all = data;
511         }
512     } else {
513         SerialA.sci->SCIFFTX.bit.TXFFIENA = 0;      // TX: disable fifo interrupt
514     }
515     SerialA.sci->SCIFFTX.bit.TXFFINTCLR = 1;  // TX: clear interrupt flag
516     PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;
517 }
518 
519 
520 //for serialB
521 #ifdef _FLASH
522 #pragma CODE_SECTION(TXBINT_data_sent, ".TI.ramfunc");
523 #endif
524 __interrupt void TXBINT_data_sent(void)
525 {
526     char data;
527     if (buf_readSCIB_1(&SerialB.TX,0,&data) == 0) {
528         while ( (buf_readSCIB_1(&SerialB.TX,0,&data) == 0)
529                 && (SerialB.sci->SCIFFTX.bit.TXFFST != 0x10) ) {
530             buf_removeSCIB(&SerialB.TX, 1);
531             SerialB.sci->SCITXBUF.all = data;
```

```
        }
    } else {
        SerialB.sci->SCIFFTX.bit.TXFFIENA = 0;      // TX: disable fifo interrupt
    }
    SerialB.sci->SCIFFTX.bit.TXFFINTCLR = 1;  // TX: clear interrupt flag
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;
}


//for serialC
#ifdef _FLASH
#pragma CODE_SECTION(TXCINT_data_sent, ".TI.ramfunc");
#endif
__interrupt void TXCINT_data_sent(void)
{
    char data;
    if (buf_readSCIC_1(&SerialC.TX,0,&data) == 0) {
        while ( (buf_readSCIC_1(&SerialC.TX,0,&data) == 0)
                && (SerialC.sci->SCIFFTX.bit.TXFFST != 0x10) ) {
            buf_removeSCIC(&SerialC.TX, 1);
            SerialC.sci->SCITXBUF.all = data;
        }
    } else {
        SerialC.sci->SCIFFTX.bit.TXFFIENA = 0;      // TX: disable fifo interrupt
    }
    SerialC.sci->SCIFFTX.bit.TXFFINTCLR = 1;  // TX: clear interrupt flag
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP8;
}

//for serialD
#ifdef _FLASH
#pragma CODE_SECTION(TXDINT_data_sent, ".TI.ramfunc");
#endif
__interrupt void TXDINT_data_sent(void)
{
    char data;
    if (buf_readSCID_1(&SerialD.TX,0,&data) == 0) {
        while ( (buf_readSCID_1(&SerialD.TX,0,&data) == 0)
                && (SerialD.sci->SCIFFTX.bit.TXFFST != 0x10) ) {
            buf_removeSCID(&SerialD.TX, 1);
            SerialD.sci->SCITXBUF.all = data;
        }
    } else {
        SerialD.sci->SCIFFTX.bit.TXFFIENA = 0;      // TX: disable fifo interrupt
    }
    SerialD.sci->SCIFFTX.bit.TXFFINTCLR = 1;  // TX: clear interrupt flag
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP8;
}

//for SerialA
#ifdef _FLASH
#pragma CODE_SECTION(RXAINT_recv_ready, ".TI.ramfunc");
#endif
__interrupt void RXAINT_recv_ready(void)
{
    RXAdata = SciaRegs.SCIRXBUF.all;

    /* SCI PE or FE error */
    if (RXAdata & 0xC000) {
        SciaRegs.SCICTL1.bit.SWRESET = 0;
        SciaRegs.SCICTL1.bit.SWRESET = 1;
        SciaRegs.SCIFFRX.bit.RXFIFORESET = 0;
        SciaRegs.SCIFFRX.bit.RXFIFORESET = 1;
    } else {
        RXAdata = RXAdata & 0x00FF;

        numRXA ++;

        if(RXAdata == 'a'){
            GpioDataRegs.GPBSET.bit.GPIO34 = 1;
        }
        else if(RXAdata == 'b'){
            GpioDataRegs.GPBCLEAR.bit.GPIO34 = 1;
        }
    }

    SciaRegs.SCIFFRX.bit.RXFFINTCLR = 1;
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;

}

//for SerialB
#ifdef _FLASH
#pragma CODE_SECTION(RXBINT_recv_ready, ".TI.ramfunc");
#endif
__interrupt void RXBINT_recv_ready(void)
{
    RXBdata = ScibRegs.SCIRXBUF.all;

```

```
621     /* SCI PE or FE error */
622     if (RXBdata & 0xC000) {
623         ScibRegs.SCICTL1.bit.SWRESET = 0;
624         ScibRegs.SCICTL1.bit.SWRESET = 1;
625         ScibRegs.SCIFFRX.bit.RXFIFORESET = 0;
626         ScibRegs.SCIFFRX.bit.RXFIFORESET = 1;
627     } else {
628         RXBdata = RXBdata & 0x00FF;
629         // Do something with recieved character
630         numRXB ++;
631     }
632 
633     ScibRegs.SCIFFRX.bit.RXFFINTCLR = 1;
634     PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;
635 }
636 
637 
638 // for SerialC
639 #ifdef _FLASH
640 #pragma CODE_SECTION(RXCINT_recv_ready, ".TI.ramfunc");
641 #endif
642 __interrupt void RXCINT_recv_ready(void)
643 {
644     RXCdata = ScicRegs.SCIRXBUF.all;
645 
646     /* SCI PE or FE error */
647     if (RXCdata & 0xC000) {
648         ScicRegs.SCICTL1.bit.SWRESET = 0;
649         ScicRegs.SCICTL1.bit.SWRESET = 1;
650         ScicRegs.SCIFFRX.bit.RXFIFORESET = 0;
651         ScicRegs.SCIFFRX.bit.RXFIFORESET = 1;
652     } else {
653         RXCdata = RXCdata & 0x00FF;
654         numRXC ++;
655 
656     }
657 
658     ScicRegs.SCIFFRX.bit.RXFFINTCLR = 1;
659     PieCtrlRegs.PIEACK.all = PIEACK_GROUP8;
660 }
661 
662 
663 //for SerialD
664 #ifdef _FLASH
665 #pragma CODE_SECTION(RXDINT_recv_ready, ".TI.ramfunc");
666 #endif
667 __interrupt void RXDINT_recv_ready(void)
668 {
669     RXDdata = ScidRegs.SCIRXBUF.all;
670 
671     /* SCI PE or FE error */
672     if (RXDdata & 0xC000) {
673         ScidRegs.SCICTL1.bit.SWRESET = 0;
674         ScidRegs.SCICTL1.bit.SWRESET = 1;
675         ScidRegs.SCIFFRX.bit.RXFIFORESET = 0;
676         ScidRegs.SCIFFRX.bit.RXFIFORESET = 1;
677     } else {
678         RXDdata = RXDdata & 0x00FF;
679             numRXD ++;
680 
681     }
682 
683     ScidRegs.SCIFFRX.bit.RXFFINTCLR = 1;
684     PieCtrlRegs.PIEACK.all = PIEACK_GROUP8;
685 }
686 
687 
688 // SerialA only setup for Tera Term connection
689 char serial_printf_bufSCIA[BUF_SIZESCIA];
690 
691 uint16_t serial_printf(serialSCIA_t *s, char *fmt, ...)
692 {
693     va_list ap;
694 
695     va_start(ap,fmt);
696     vsprintf(serial_printf_bufSCIA,fmt,ap);
697     va_end(ap);
698 
699     return serial_sendSCIA(s,serial_printf_bufSCIA,strlen(serial_printf_bufSCIA));
700 }
```